AWS VPC Peering

Connect two VPCs and prove private connectivity

Field	Details
Name	Nithin N
Batch	AWS Internship Batch 1
Project	Project 01: VPC Peering (EC2, route tables, security groups, bastion host)
Region	Asia Pacific (Mumbai), ap-south-1
Completion date	23 October 2026
Organisation	EWB EDUTECH Private Limited (AWS Cloud Computing Internship, via AICTE)

1. Project summary

This project connects two isolated Amazon VPCs, VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16), with a VPC peering connection so that two private servers can exchange traffic using only their private IPv4 addresses. A single public bastion host in VPC A is the only entry point for administration. Neither application server has a public IP address.

The work shows a failed test before peering, successful two-way tests after peering, and a controlled failure caused by removing one security group rule and then restoring it.

Architecture

Administration path: laptop > bastion (public subnet, VPC A) > App-A or App-B using SSH ProxyJump. Project traffic path: App-A <> App-B over private IPs through the peering connection. VPC B has no internet gateway.

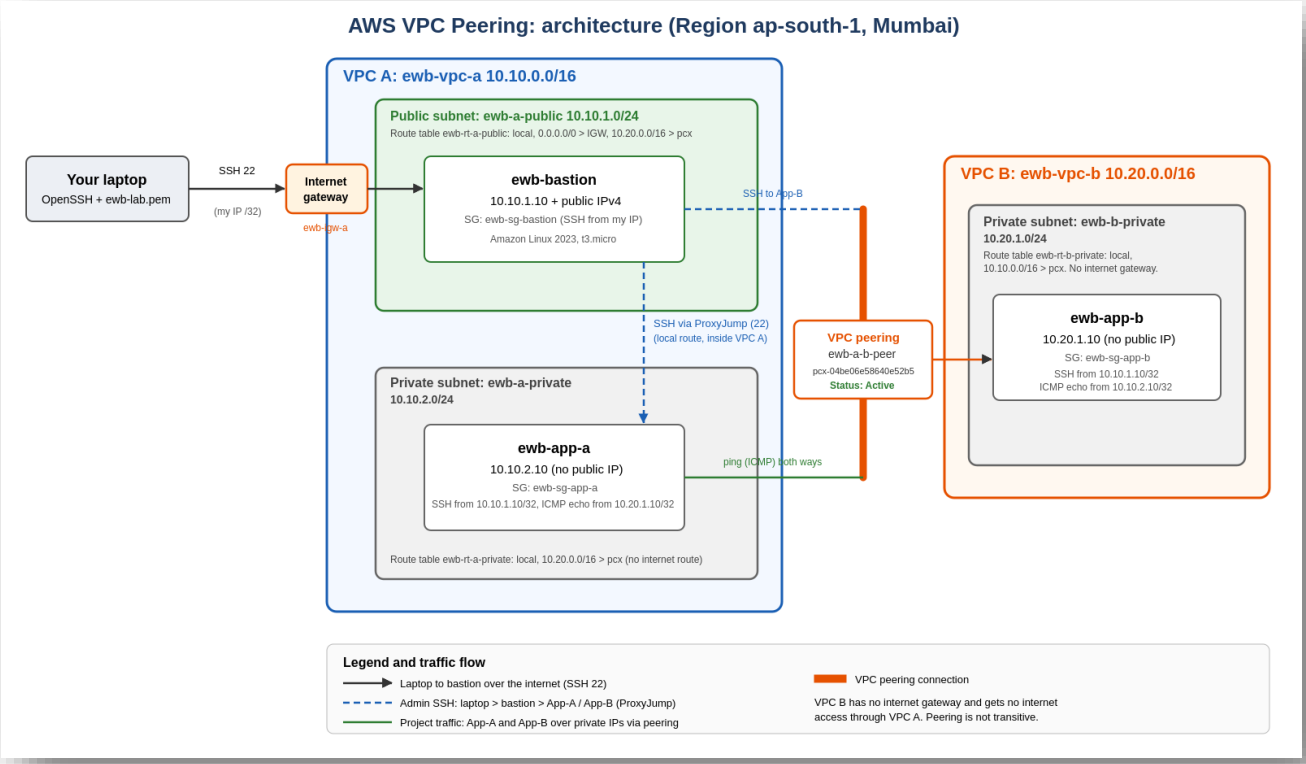


Figure: Architecture diagram (draw it in draw.io, PowerPoint or by hand, showing both VPCs, three subnets, the IGW, the bastion, App-A, App-B and the peering link)

2. IP plan and resource inventory

Resource	Location	CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A, public subnet	10.10.1.0/24
ewb-a-private	VPC A, private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B, private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4
ewb-app-a	ewb-a-private	10.10.2.10, no public IP
ewb-app-b	ewb-b-private	10.20.1.10, no public IP

The two VPC ranges do not overlap, which is a requirement for peering. Each /24 subnet sits inside its own VPC range.

Actual resource IDs (Region ap-south-1)

Resource	Name tag	ID
VPC A	ewb-vpc-a	`vpc-08d560a97471548a6`
VPC B	ewb-vpc-b	`vpc-0f4759c8172c2be0d`
Subnet (public, A)	ewb-a-public	`subnet-01ee83d31ebbc1839`
Subnet (private, A)	ewb-a-private	`subnet-0d971edd4ef91bf78`
Subnet (private, B)	ewb-b-private	`subnet-0a443f86d4c11b4e2`
Internet gateway	ewb-igw-a (attached to VPC A)	`igw-0df3f159a374301f5`
Route table	ewb-rt-a-public	`rtb-073ad0bb135111296`
Route table	ewb-rt-a-private	`rtb-0db5e5323ba5618b1`
Route table	ewb-rt-b-private	`rtb-0880b38b4b6db5027`
Security group	ewb-sg-bastion	`sg-09e1cc32e7e02f3f7`
Security group	ewb-sg-app-a	`sg-0ff531122bfb8633b`
Security group	ewb-sg-app-b	`sg-020f6274cc7f2d15a`
Peering connection	ewb-a-b-peer	`pcx-04be06e58640e52b5`
EC2 instance	ewb-bastion (10.10.1.10, public 13.201.123.205)	`i-0d18e6e20b165e368`
EC2 instance	ewb-app-a (10.10.2.10)	`i-03d766be732b9ab33`
EC2 instance	ewb-app-b (10.20.1.10)	`i-05d24e7385f11ce80`

All resources were tagged Project = EWB-VPC-Peering and Student = Nithin N. Every resource listed above was deleted during cleanup (section 13).

3. VPC A: subnets, internet gateway and routing

I created ewb-vpc-a with the VPC only option, then two subnets in the same Availability Zone (ap-south-1a). The internet gateway ewb-igw-a was attached to VPC A. A subnet is public only because its route table has a 0.0.0.0/0 route to the internet gateway.

Route table	Associated subnet	Routes (before peering)
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 > local; 0.0.0.0/0 > igw-0df3f159a374301f5
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 > local only (no internet route)

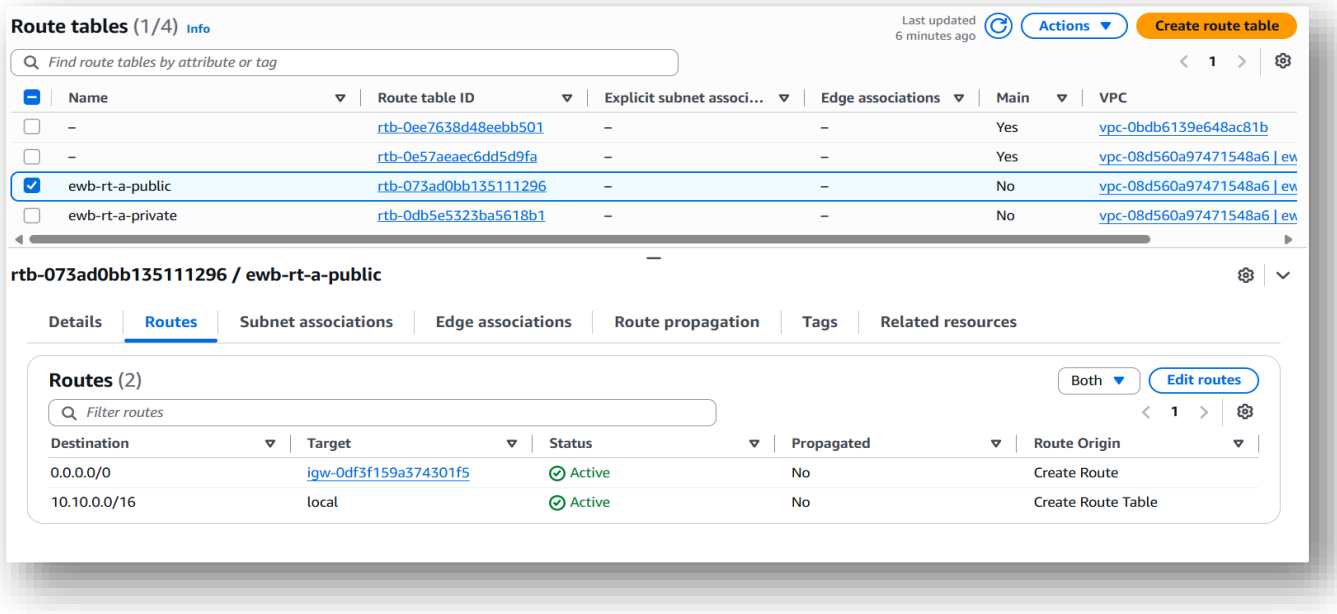


Figure: ewb-rt-a-public, Routes tab

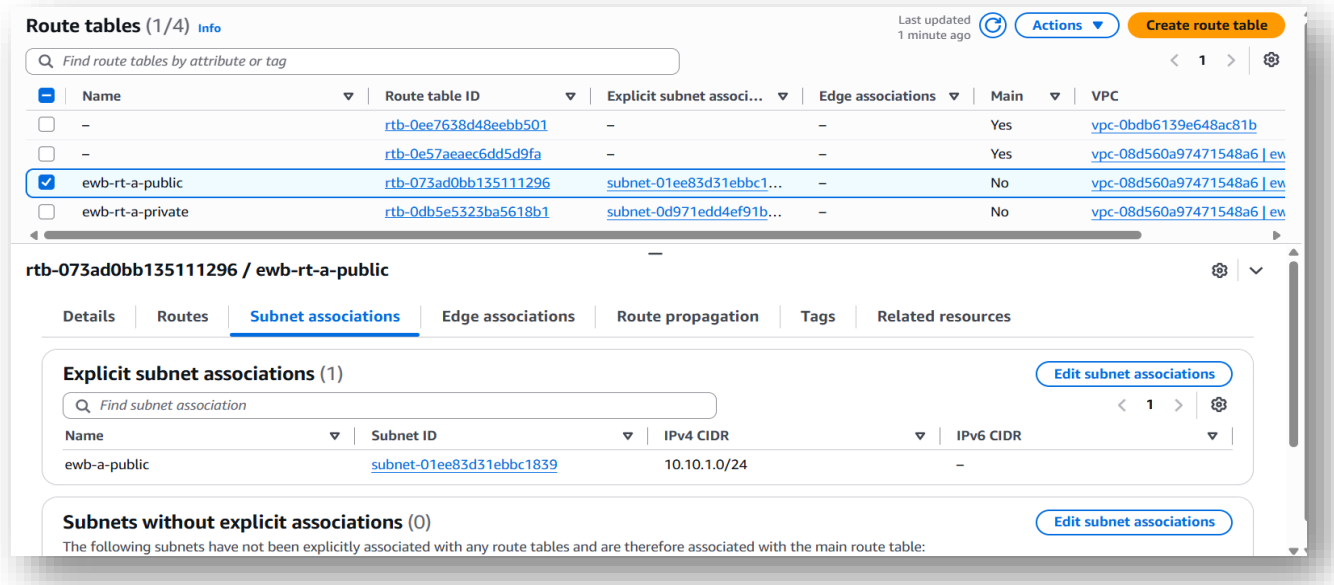


Figure: ewb-rt-a-public, Subnet associations tab

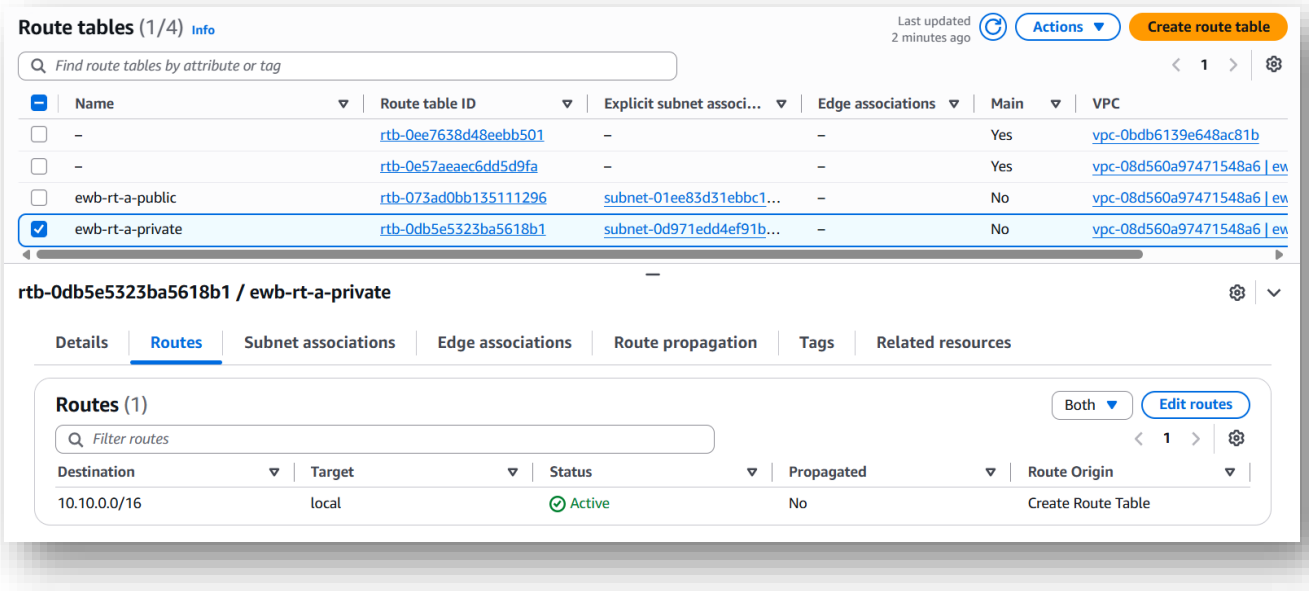


Figure: ewb-rt-a-private, Routes tab

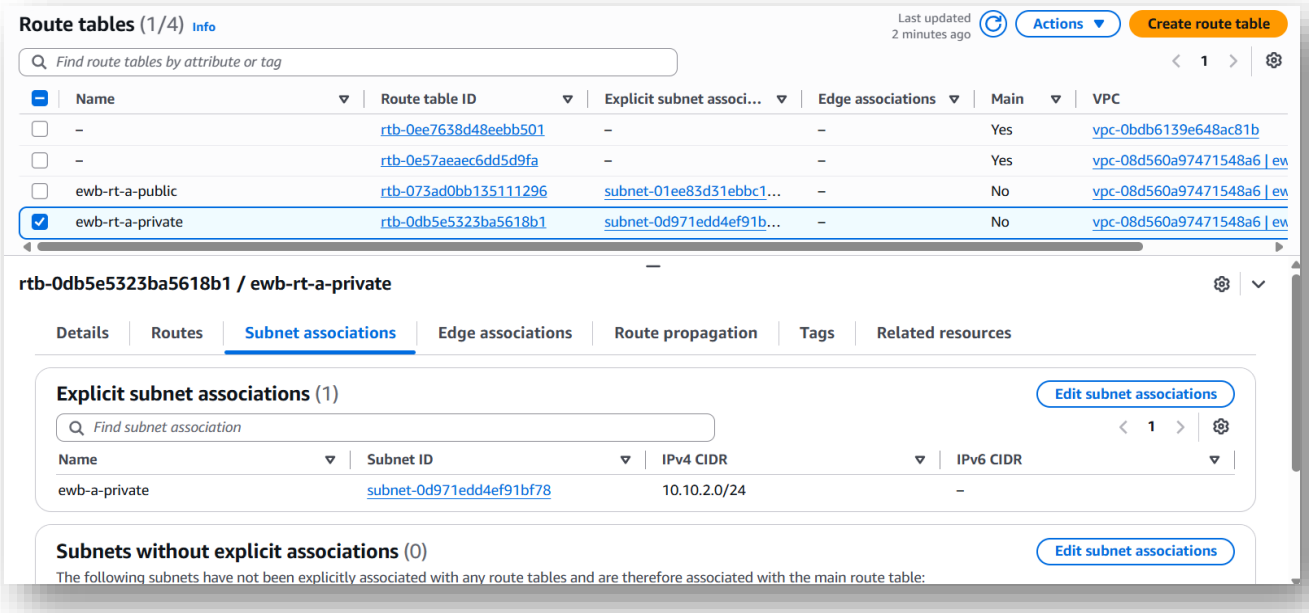


Figure: ewb-rt-a-private, Subnet associations tab

4. VPC B and the full network

ewb-vpc-b (10.20.0.0/16) has one private subnet, ewb-b-private (10.20.1.0/24), with route table ewb-rt-b-private explicitly associated. VPC B has no internet gateway.

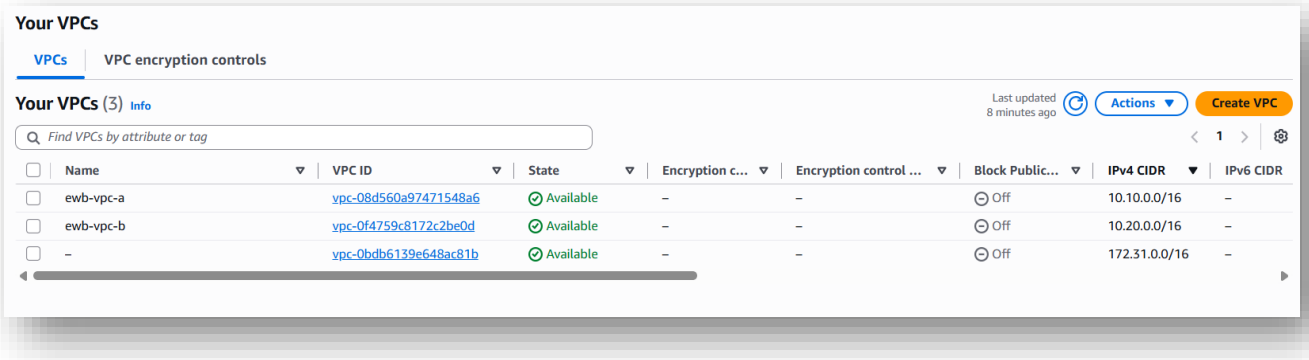


Figure: Your VPCs: ewb-vpc-a and ewb-vpc-b with their CIDRs

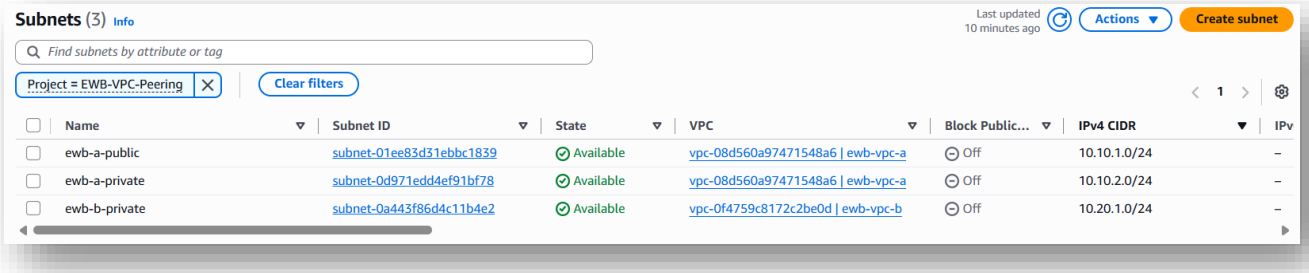


Figure: Subnets: all three project subnets with CIDRs and VPCs

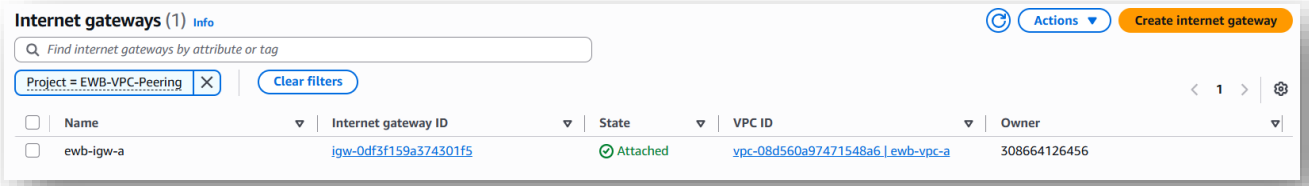
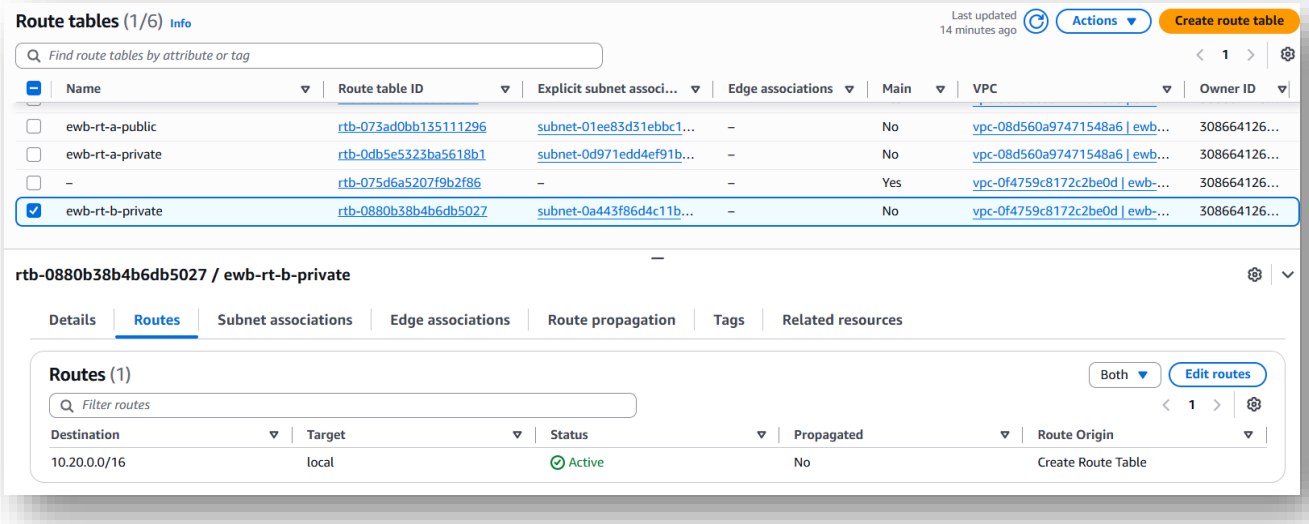


Figure: ewb-igw-a attached to ewb-vpc-a



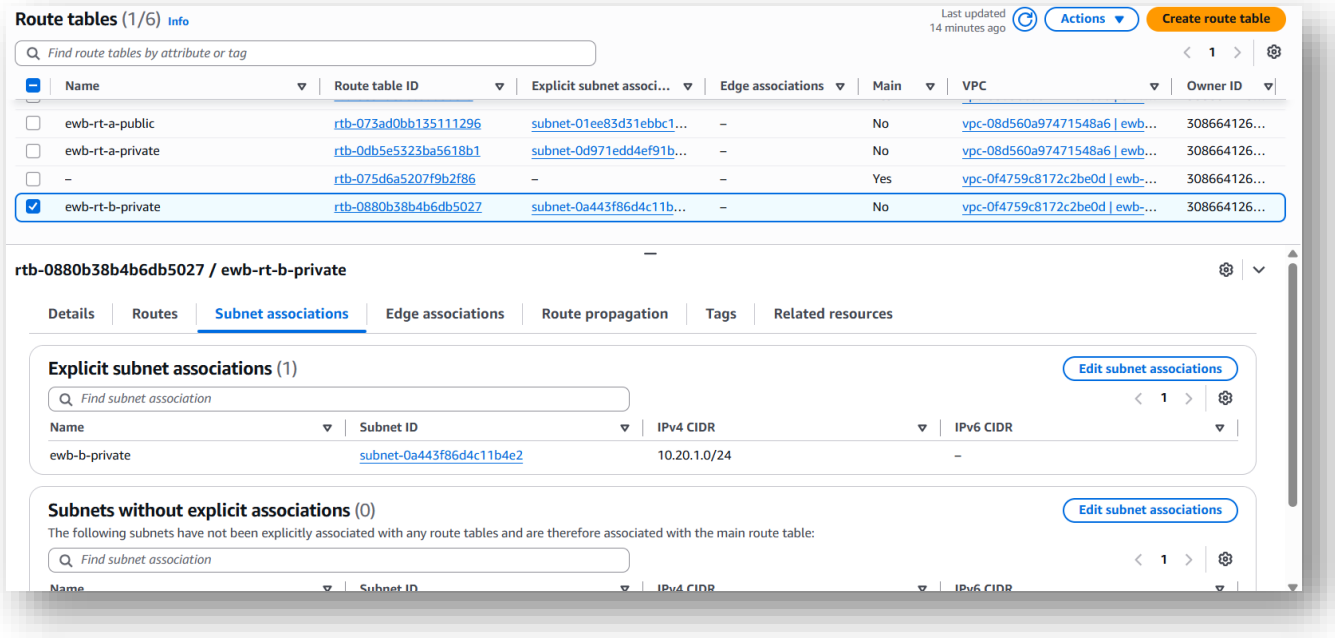


Figure: ewb-rt-b-private: local route and association with ewb-b-private

5. Security groups

Outbound rules were left at the default (allow all). Security groups are stateful, so replies to allowed traffic are returned automatically. No rule uses 0.0.0.0/0.

Security group (VPC)	Type / port	Source
ewb-sg-bastion (A)	SSH / TCP 22	My public IPv4 /32
ewb-sg-app-a (A)	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a (A)	ICMP Echo request (type 8)	10.20.1.10/32 (App-B)
ewb-sg-app-b (B)	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b (B)	ICMP Echo request (type 8)	10.10.2.10/32 (App-A)

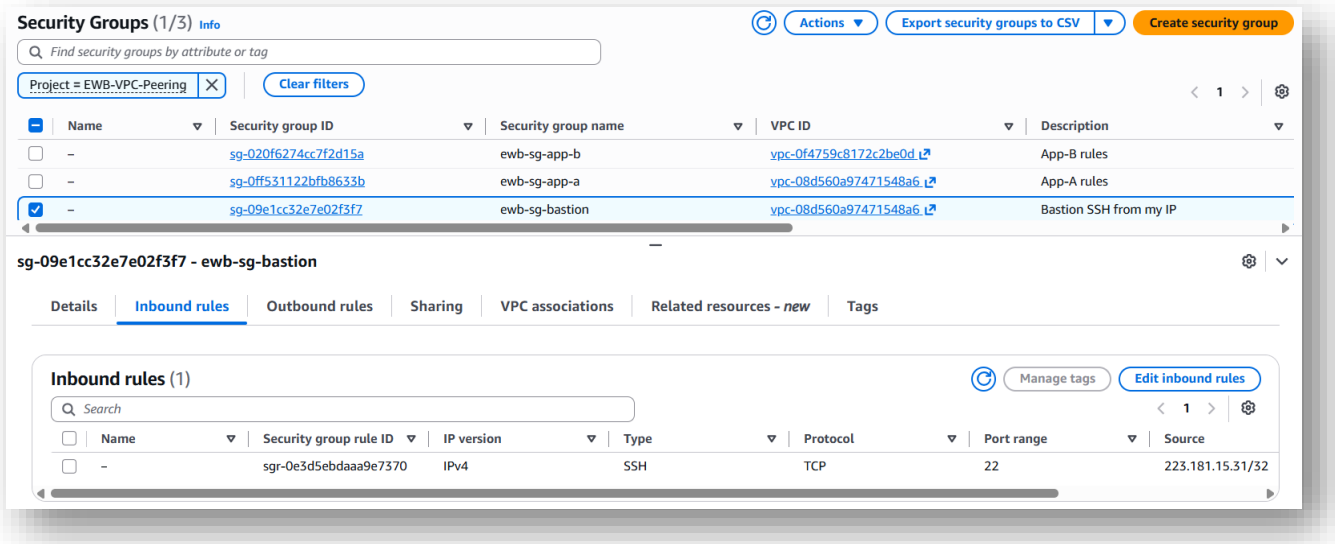


Figure: ewb-sg-bastion, inbound rules (VPC ID visible)

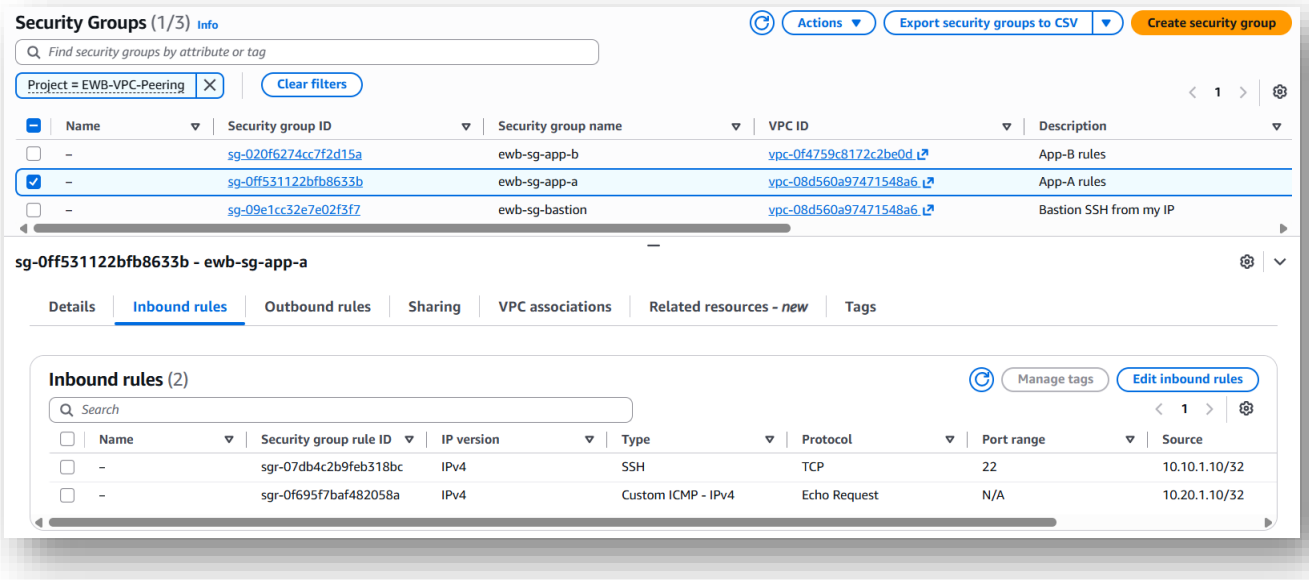


Figure: ewb-sg-app-a, inbound rules

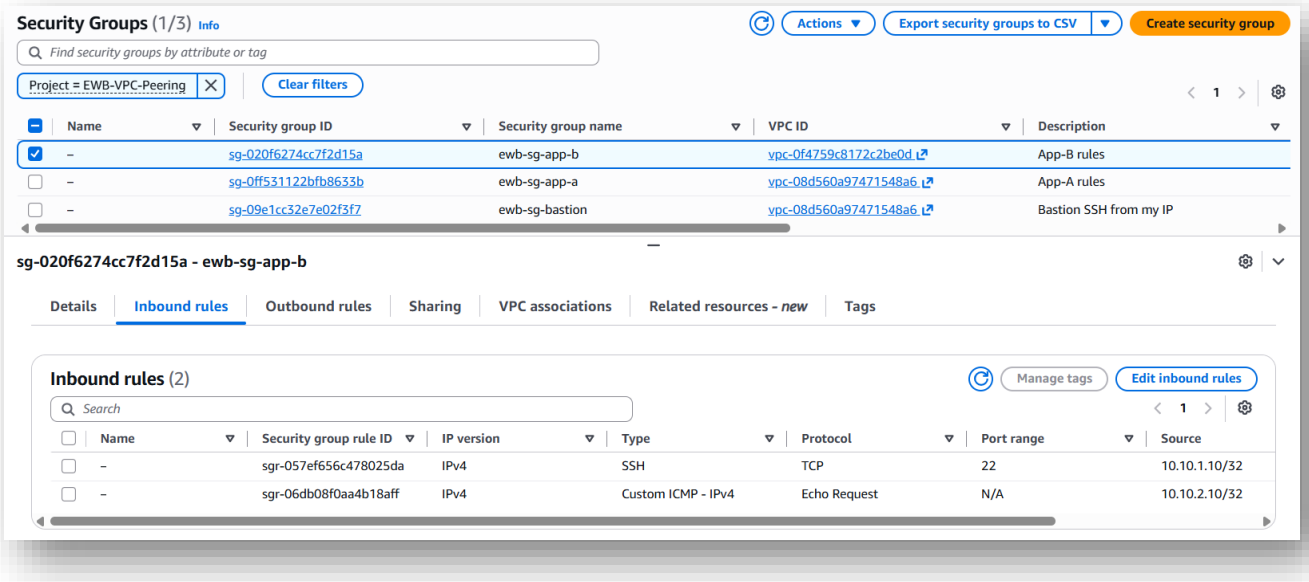


Figure: ewb-sg-app-b, inbound rules

6. EC2 instances

Three Amazon Linux 2023 t3.micro instances were launched with key pair ewb-lab and fixed private IPs. Only the bastion received a public IPv4 address (13.201.123.205). App-A and App-B show no public IPv4 address.

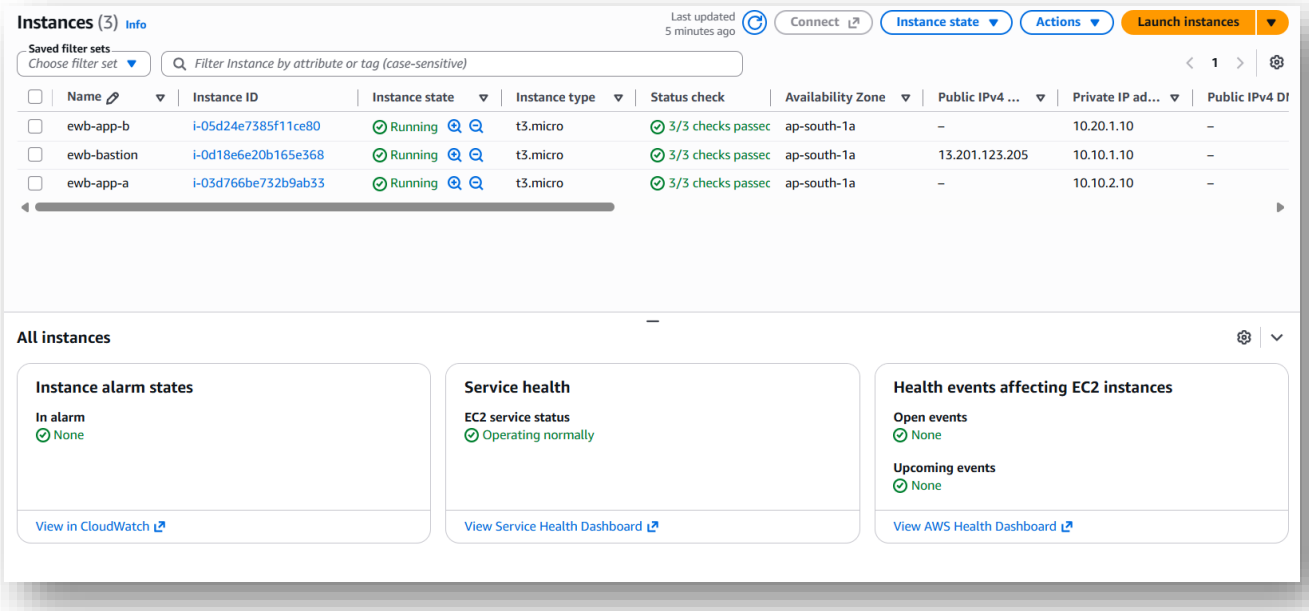


Figure: Instances list: IDs, state, private IPv4 addresses

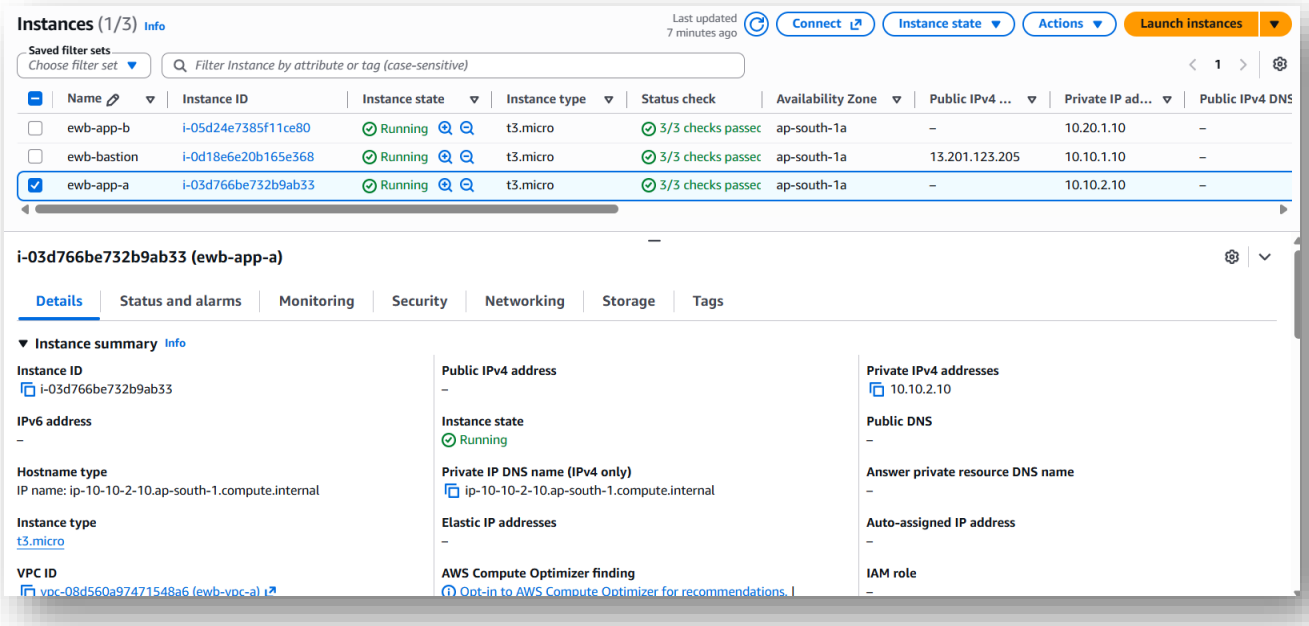


Figure: ewb-app-a details: Public IPv4 address empty

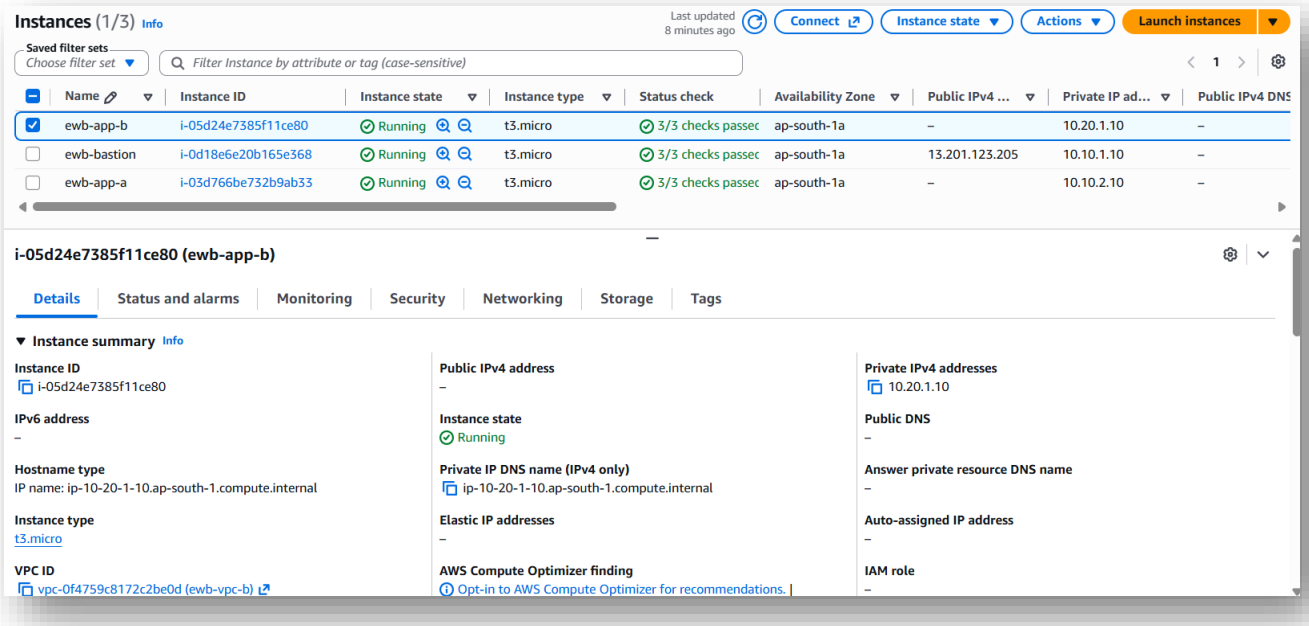


Figure: ewb-app-b details: Public IPv4 address empty

7. SSH access and baseline test

I used the Windows OpenSSH client with a config file that defines ewb-bastion, ewb-app-a and ewb-app-b. App-A and App-B use ProxyJump through the bastion, so the private key stays on my laptop and is never copied to any server. The private key is not included in this report.

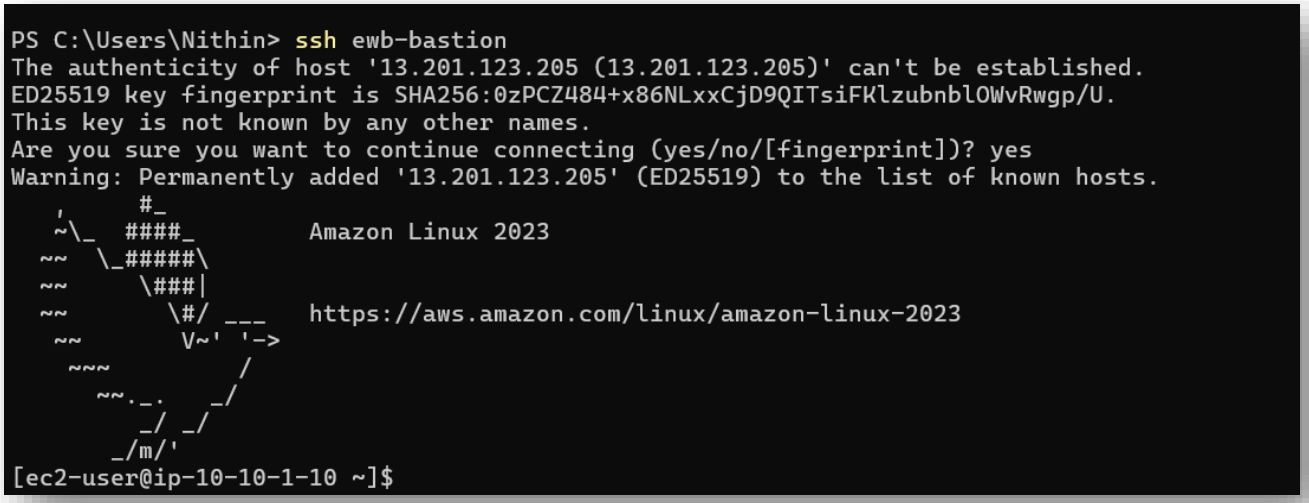


Figure: Laptop > ewb-bastion: SSH succeeded (prompt shows ip-10-10-1-10)

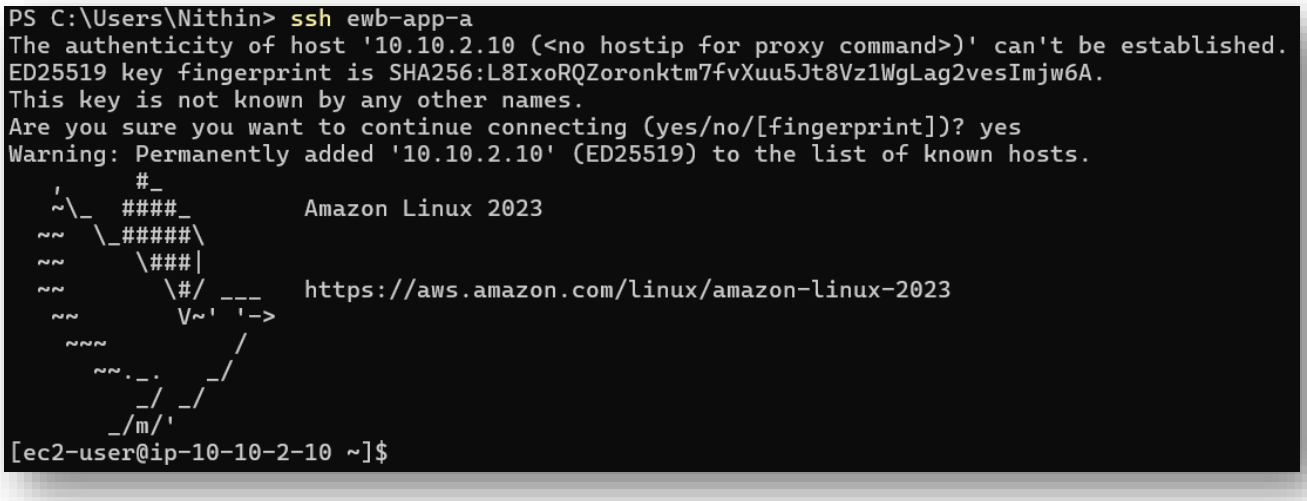


Figure: Laptop > ewb-app-a via bastion: SSH succeeded (prompt shows ip-10-10-2-10)

Test 01: Baseline (before peering routes)

From App-A: hostname, hostname -I, ping -c 4 10.20.1.10. Result: 100% packet loss, because VPC A has no route to 10.20.0.0/16 and no peering connection existed yet.

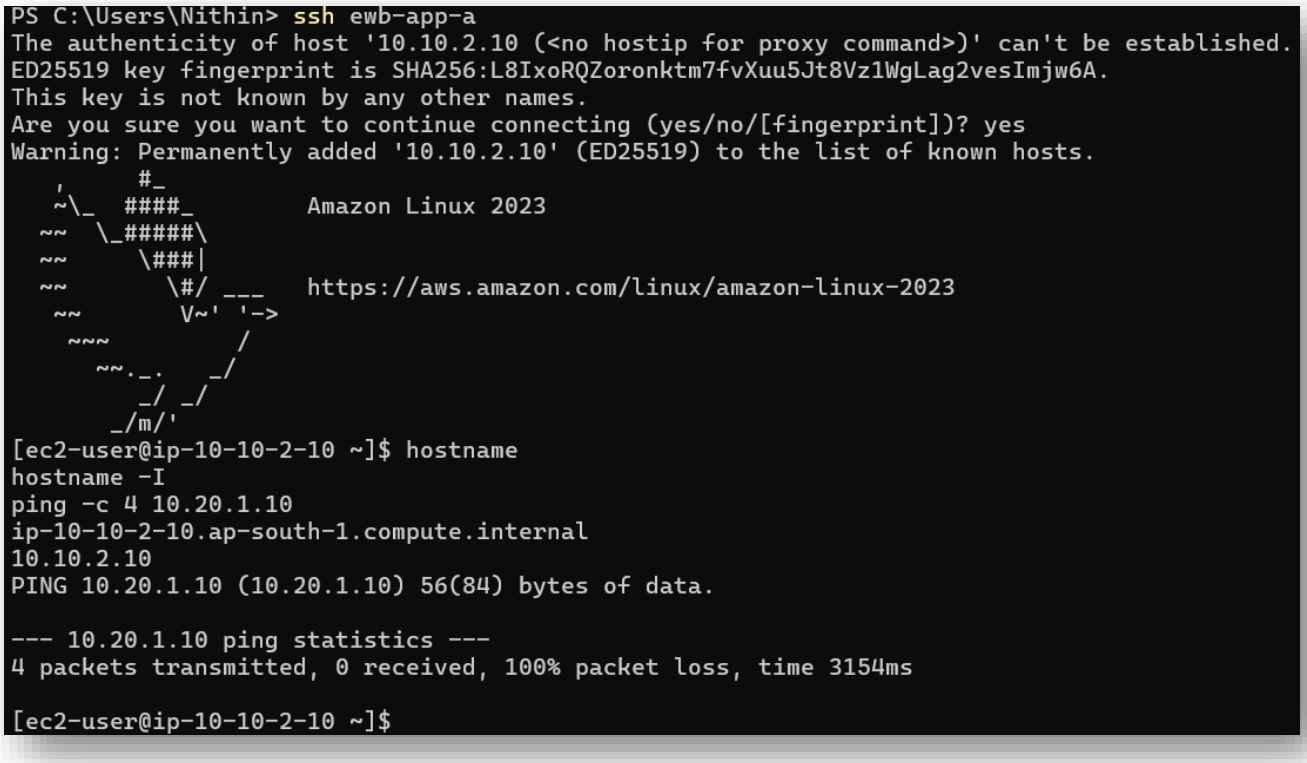


Figure: Host: App-A | Test 01 baseline | Result: no replies

8. VPC peering connection

I created peering connection ewb-a-b-peer (pcx-04be06e58640e52b5) with VPC A as requester and VPC B as accepter, in the same account and Region, then accepted the request. Status: Active.

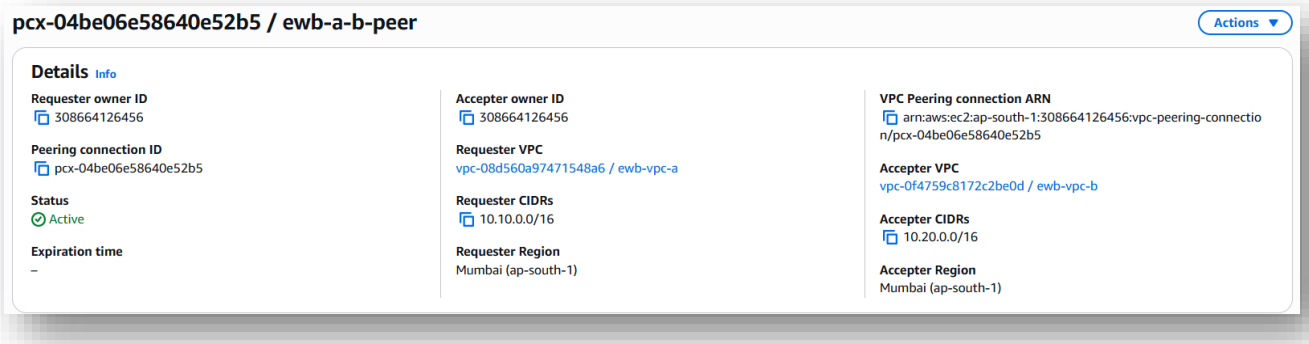


Figure: ewb-a-b-peer: Active, requester VPC A, accepter VPC B

Peering Active, route removed (extra evidence)

[Include this part only if you took screenshot 15b. Otherwise delete this heading, this paragraph and the figure below.]

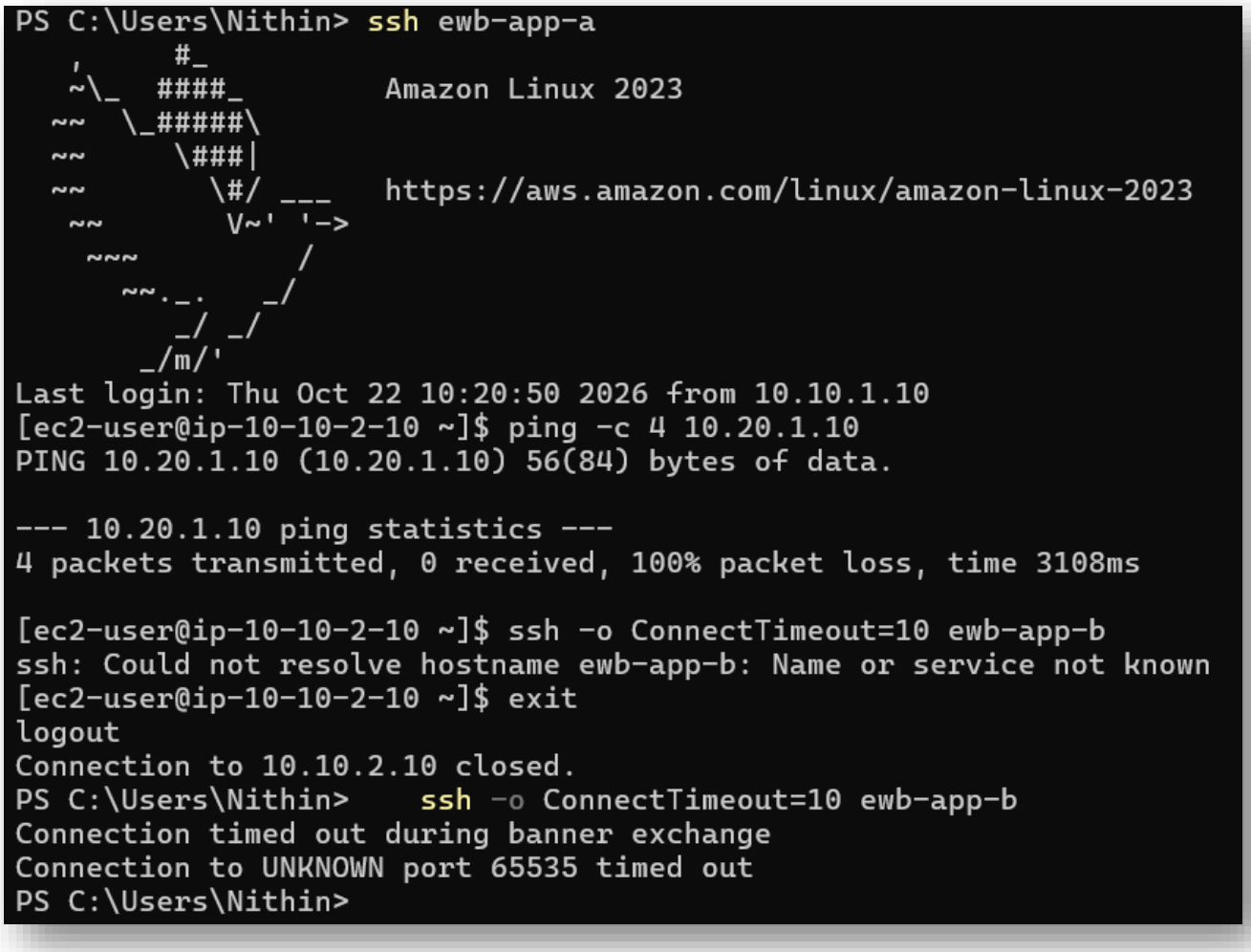


Figure: Host: App-A | Peering Active but the peer route temporarily removed | Result: no replies. Route restored afterwards.

Active peering alone does not make traffic flow. Routes on both sides and the destination security group rule are also required.

9. Routes after peering

Route table	Destination	Target	Purpose
ewb-rt-a-private	10.10.0.0/16	local	Existing VPC A route
ewb-rt-a-private	10.20.0.0/16	pcx-04be06e58640e52b5	App-A to VPC B
ewb-rt-b-private	10.20.0.0/16	local	Existing VPC B route
ewb-rt-b-private	10.10.0.0/16	pcx-04be06e58640e52b5	Return path to VPC A
ewb-rt-a-public	10.10.0.0/16	local	Bastion to App-A
ewb-rt-a-public	0.0.0.0/0	igw-0df3f159a374301f5	Bastion internet access
ewb-rt-a-public	10.20.0.0/16	pcx-04be06e58640e52b5	Bastion SSH to App-B

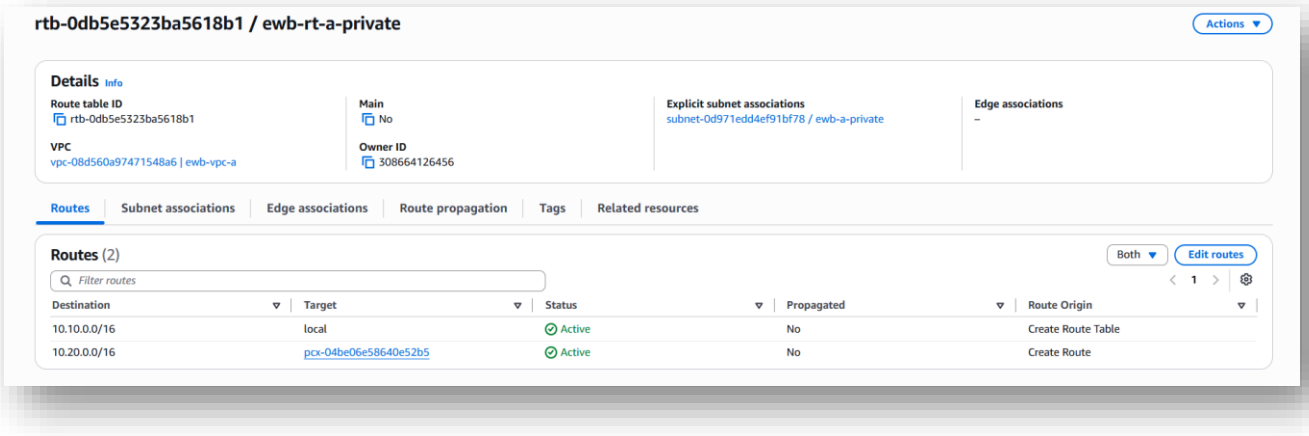


Figure: ewb-rt-a-private with peer route

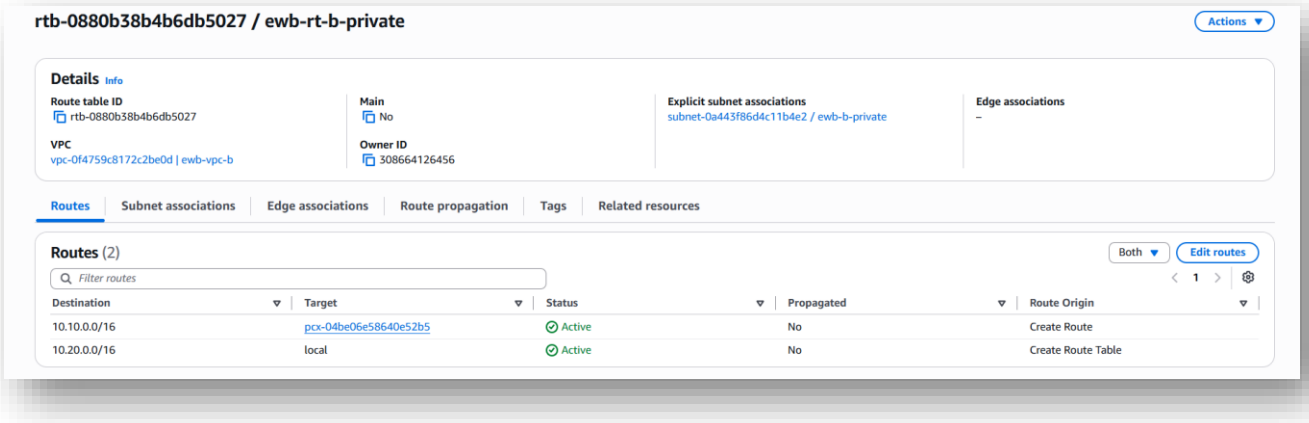


Figure: ewb-rt-b-private with return route

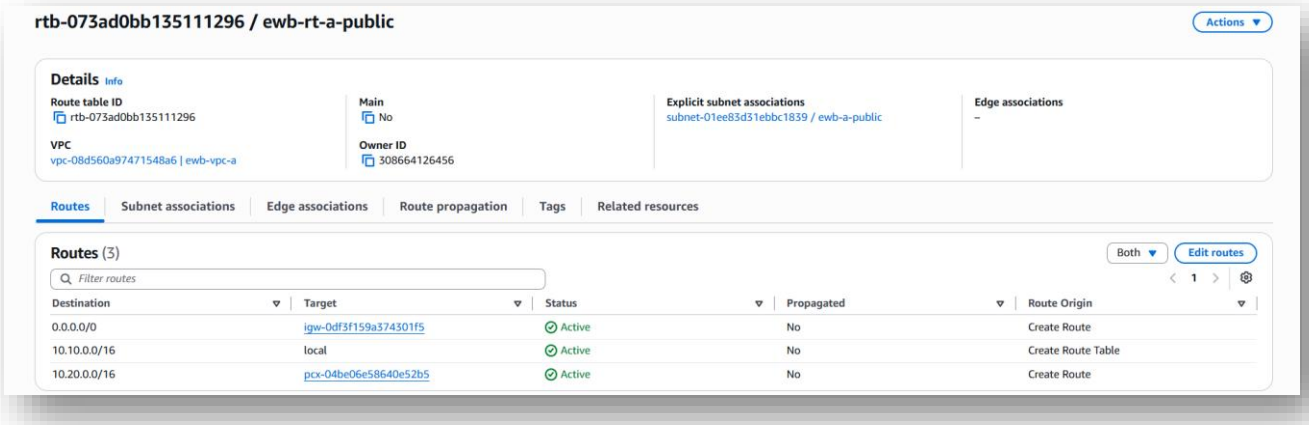


Figure: ewb-rt-a-public with peer route

10. Verification tests

Expected results are acceptance criteria. The outputs below are my own execution results.

Test	Where / command	Expected	Result
01 Baseline	App-A, before peering: ping -c 4 10.20.1.10	No replies	(fill in: e.g. 100% packet loss)
02 Forward path	App-A: ping -c 4 10.20.1.10	4 replies, 0% loss	(fill in from your output)
03 Reverse path	App-B: ping -c 4 10.10.2.10	4 replies, 0% loss	(fill in from your output)
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	(fill in)
05 Isolation	EC2 details of App-A and App-B	No public IPv4	Confirmed (figures 10 and 11)

Test 02: Forward path (App-A to App-B)

```
PS C:\Users\Nithin> ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Fri Oct 23 10:27:07 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ hostname
ip-10-10-2-10.ap-south-1.compute.internal
[ec2-user@ip-10-10-2-10 ~]$ hostname -I
10.10.2.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.500 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.192 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.209 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.222 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3154ms
rtt min/avg/max/mdev = 0.192/0.280/0.500/0.127 ms
[ec2-user@ip-10-10-2-10 ~]$ exit
logout
Connection to 10.10.2.10 closed.
```

Figure: Host: App-A | Test 02 forward ping | Result: replies received

Test 04: Private SSH to App-B through the bastion

```
PS C:\Users\Nithin> ssh ewb-app-b
The authenticity of host '10.20.1.10 (<no hostip for proxy command>)' can't be establi
shed.
ED25519 key fingerprint is SHA256:3ESPHppumA9zGKJMg9M3GMyj1oXvn6q7HQB9p6EsbeE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.20.1.10' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-20-1-10 ~]$
```

Figure: Laptop > bastion > ewb-app-b | Test 04 | Result: shell on ip-10-20-1-10

Test 03: Reverse path (App-B to App-A)

```
PS C:\Users\Nithin> ssh ewb-app-b
The authenticity of host '10.20.1.10 (<no hostip for proxy command>)' can't be establi
shed.
ED25519 key fingerprint is SHA256:3ESPHppumA9zGKJMg9M3GMyj1oXvn6q7HQB9p6EsbeE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.20.1.10' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-20-1-10 ~]$ hostname
hostname -I
ping -c 4 10.10.2.10
ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.226 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.195 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.197 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.215 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3138ms
rtt min/avg/max/mdev = 0.195/0.208/0.226/0.012 ms
[ec2-user@ip-10-20-1-10 ~]$ exit
logout
Connection to 10.20.1.10 closed.
```

Figure: Host: App-B | Test 03 reverse ping | Result: replies received

11. Controlled failure and recovery

I removed only the ICMP Echo request rule (source 10.10.2.10/32) from ewb-sg-app-b and kept the SSH rule. The ping from App-A stopped. I then restored the exact same rule and the replies returned.

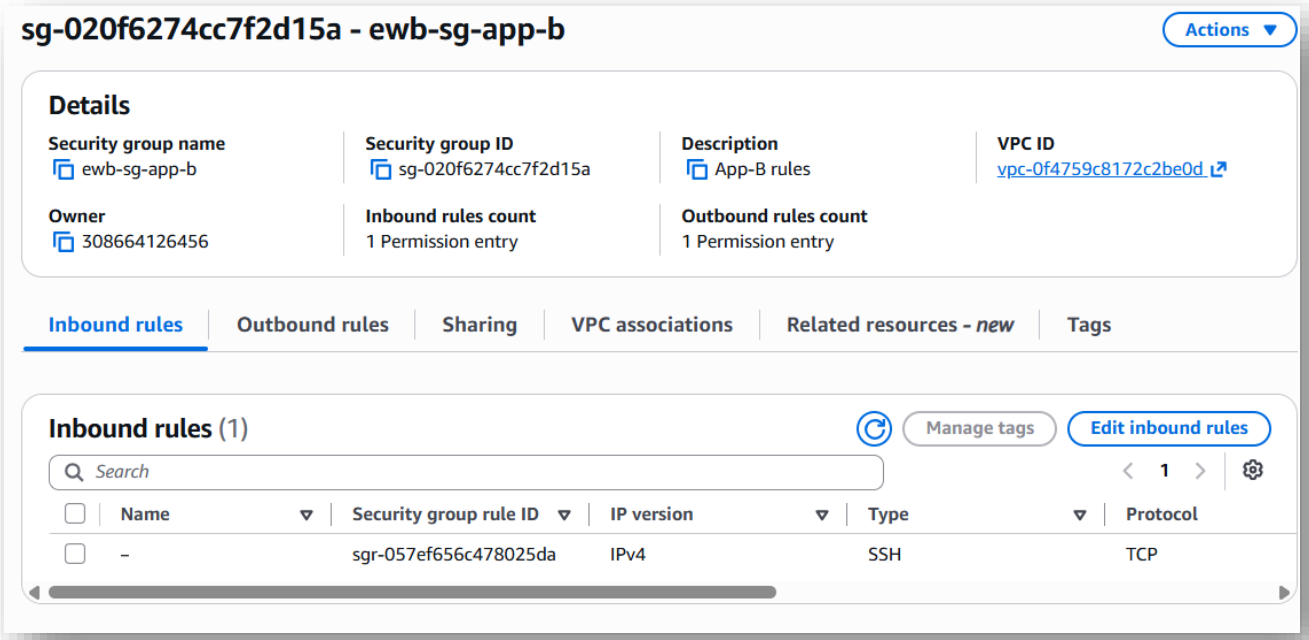


Figure: ewb-sg-app-b: ICMP rule removed, SSH rule kept

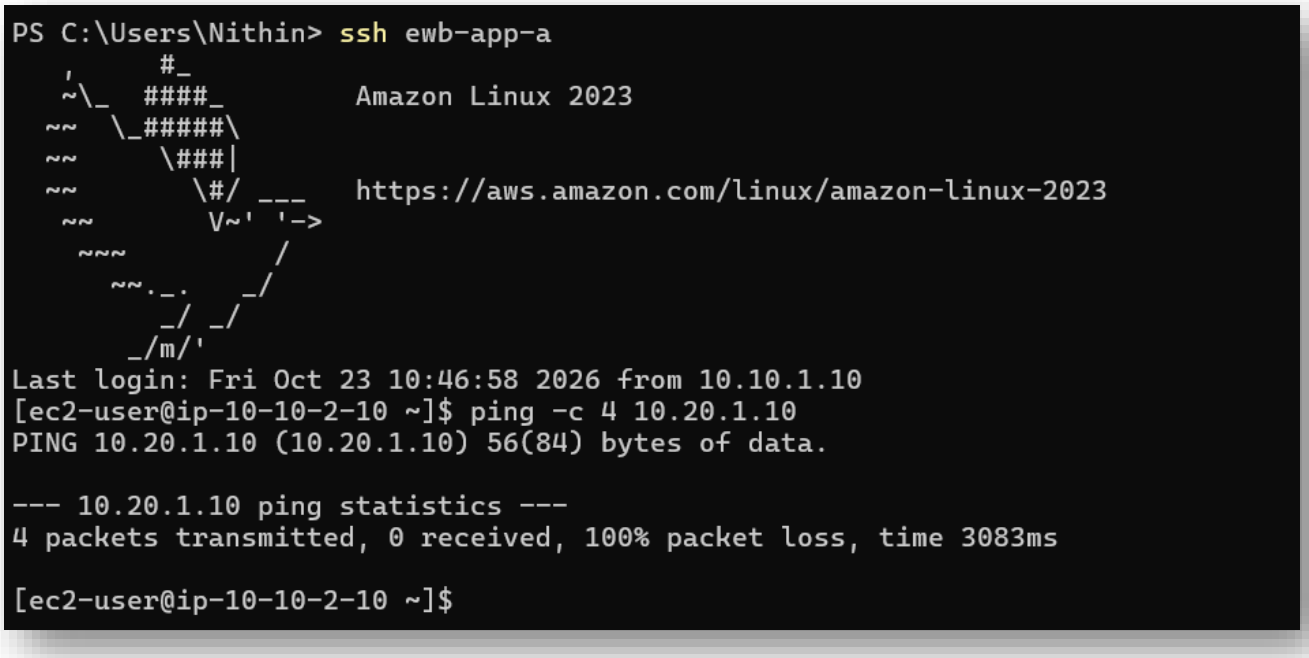


Figure: Host: App-A | ping after rule removal | Result: no replies

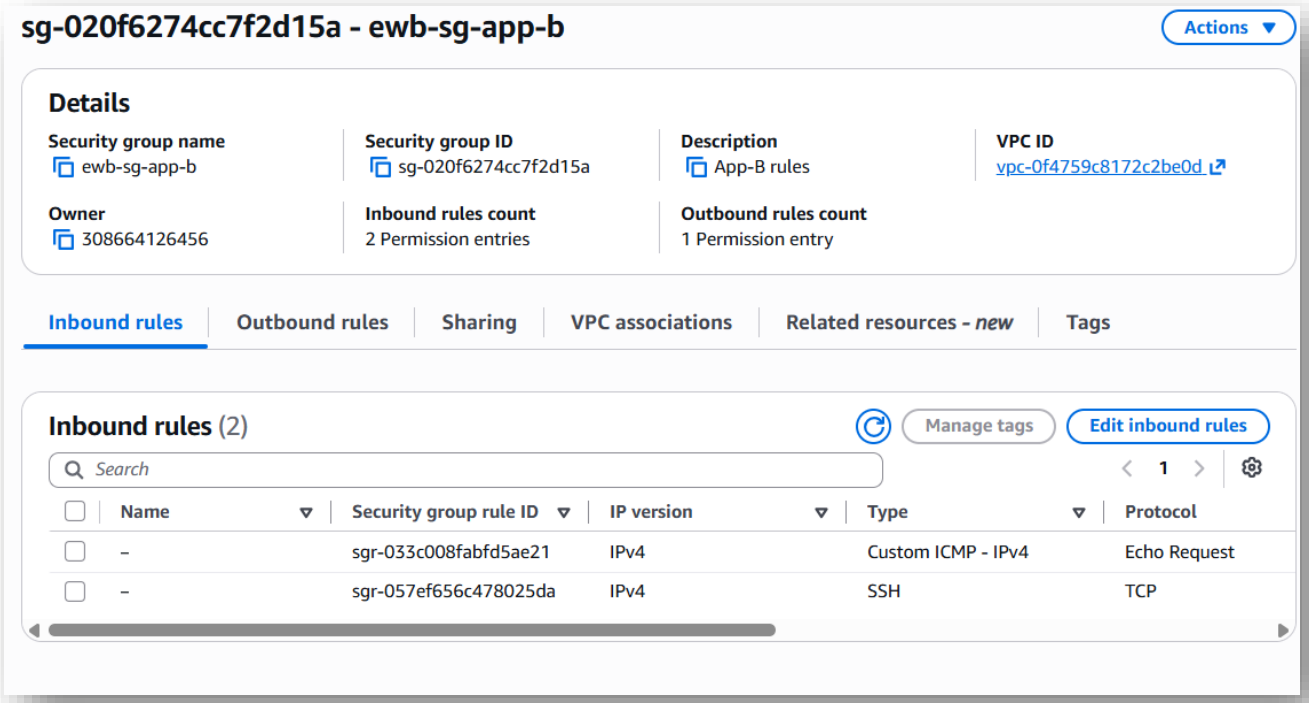


Figure: ewb-sg-app-b: ICMP rule restored

```
PS C:\Users\Nithin> ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Fri Oct 23 10:46:58 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3083ms

[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.247 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.193 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.194 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.203 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3095ms
rtt min/avg/max/mdev = 0.193/0.209/0.247/0.022 ms
[ec2-user@ip-10-10-2-10 ~]$ exit
logout
Connection to 10.10.2.10 closed.
```

Figure: Host: App-A | ping after rule restored | Result: replies resume

Explanation: The peering connection and routes did not change during this test, so packets still reached App-B. Without the Echo request rule, the App-B security group dropped them and no replies came back. Restoring the rule brought the replies back. Routing and security groups are separate layers, and both must allow the traffic.

12. Forward path, return path

Forward path (App-A to App-B)

App-A (10.10.2.10) sends a packet to 10.20.1.10. The ewb-rt-a-private table matches 10.20.0.0/16 and sends it to the peering connection. It arrives in VPC B, where the ewb-sg-app-b rule allows Echo request from 10.10.2.10/32.

Return path (App-B to App-A)

App-B replies to 10.10.2.10. The ewb-rt-b-private table matches 10.10.0.0/16 and sends the reply through the same peering connection. The security group is stateful, so the reply is allowed automatically.

13. Cleanup

After saving the evidence, I removed the resources in dependency order: terminated the three instances and checked that their volumes were gone, deleted the peer routes and the peering connection, deleted the security groups, subnets and custom route tables, detached and deleted ewb-igw-a, deleted both VPCs, and deleted the key pair ewb-lab plus the local .pem file. No Elastic IP was ever allocated.

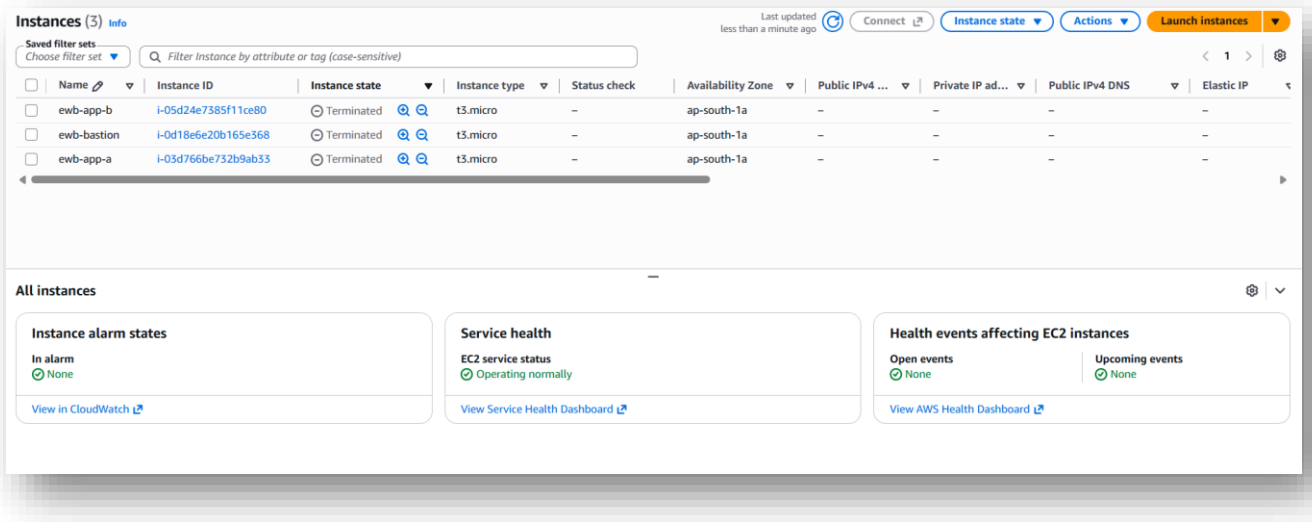


Figure: All three instances in Terminated state

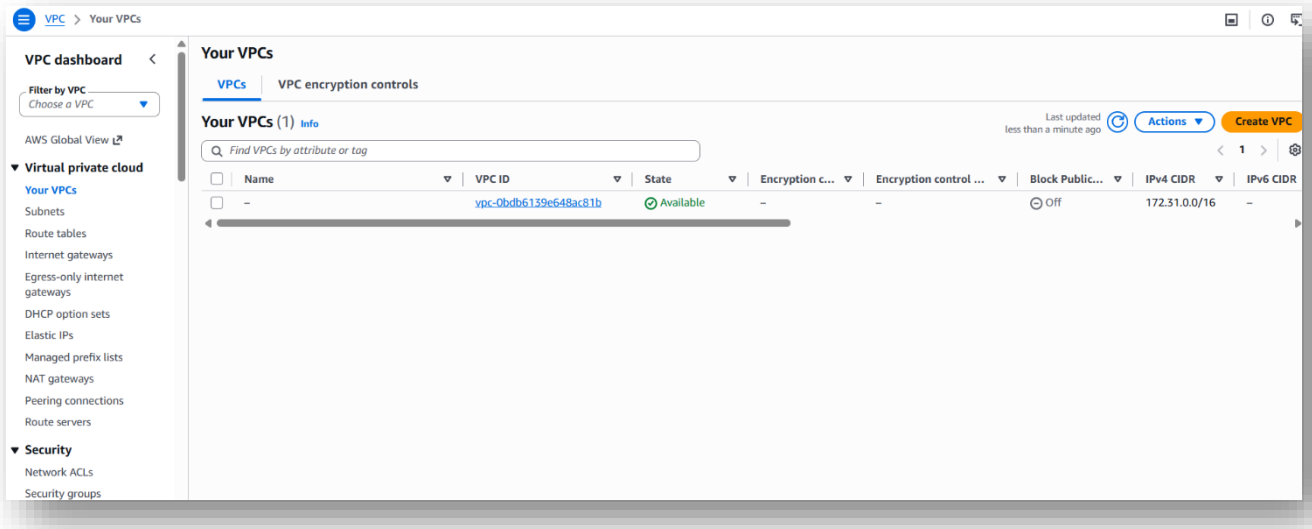


Figure: Your VPCs: only the default VPC remains

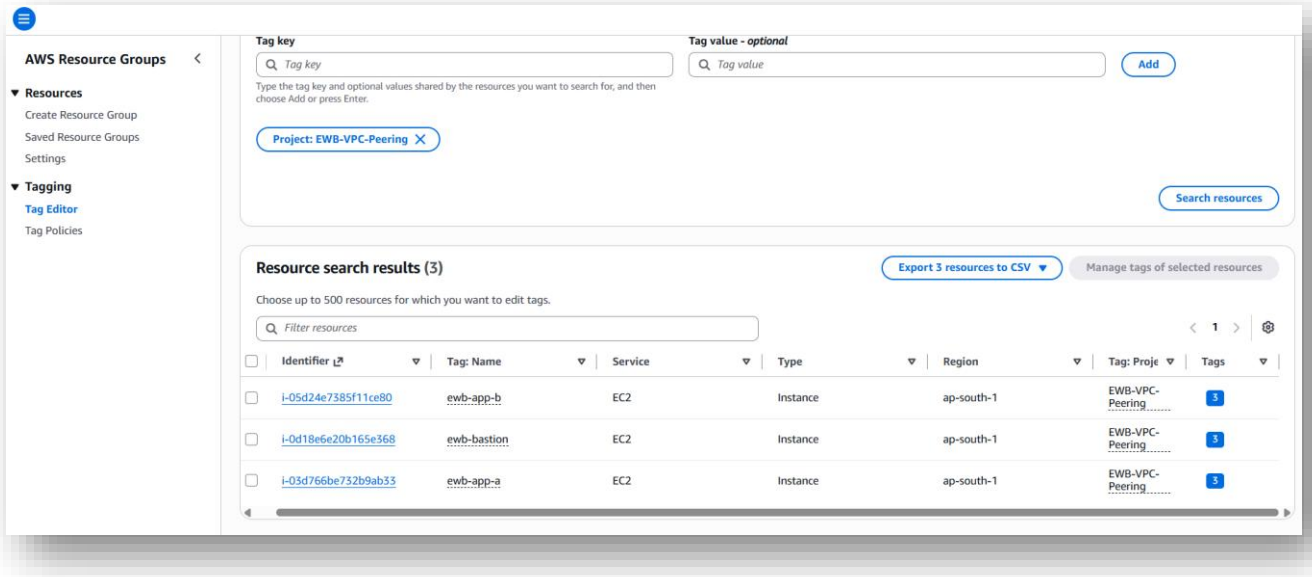


Figure: Tag Editor search for Project = EWB-VPC-Peering. Only the three already-terminated instances are still listed; AWS removes terminated instances from listings automatically within about an hour.

14. What I learned

- A VPC is an isolated network, and a subnet is public or private only because of its route table.
- Peering needs three things together: an accepted connection, routes on both sides, and security group rules on the destination.
- A bastion host with SSH ProxyJump lets me administer private servers without giving them public IPs or copying keys onto them.
- Narrow /32 security group rules are safer than opening 0.0.0.0/0, even though the class demos used Anywhere.
- Deleting resources in the right order, and checking the tag filter afterwards, avoids leftover charges.